

3Designs and Manufacturing

Daniel Magann

Kaiden Hay

4/9/2026

Website Link

<http://classwork.engr.oregonstate.edu:42565/>

Step 1 Feedback

Feedback from TA

Many of your entities missing relationship details like, colors, Materials, Machine.

Although you have a junction table you didnt define the M:N relationship in the database design layout.

Lack of numbers, you could have mentioned what is the number of current cutomers , annual sales value etc.

Please go though the database design template from the assignment very well.

Razeel Hassan Ibrahim Rasheed, Apr 16 at 1:08pm

Feedback from students

Hunter Androsenko-Hale^{4w}

Does the overview describe what problem is to be solved by a website with DB back end? If yes, summarize. If not, what changes would better support describing the problem to be solved?

Yes. The company wants a way to improve its sales based on the type of the products they are selling and what design choices are popular.

Does the overview list specific facts? If yes, summarize what the facts illustrate about the proposed DB solution. If not, what facts would better support illustrating the scope and scale of the proposed DB solution?

Yes. The database will store information about consumers' purchases so that the business can increase their inventory and sales accordingly to what sells best or most often. The company is wanting the database to specifically track type/color of the material used for the 3D print, individual parts a customer ordered, this way, the company can know what to keep in stock or have a higher base stock held for future.

Are at least four entities described, and does each one represent a single idea to be stored as a list? If yes, summarize. If not, based on the course material, what changes can you suggest to improve?

There are 6 different entities purposed, list of Customers, Orders, Part, Colors, Material, and Machine. All of them are individual and make sense to the project.

Does the outline of entity details describe the purpose of each, list attribute datatypes and constraints, and describe relationships between entities? If yes, summarize. If not, based on the course material, what changes can you suggest to improve?

Yes. All the entities have descriptions and their attributes. "Customers", "Orders", and "Part" all have relationships. "Colors", "Material", and "Machine" do not have responding relationships stated within their attribute lists to "Part" they are only stated within the "Part" entity list.

Are 1:M relationships correctly formulated? Is there at least one M:M relationship? Does the ERD present a logical view of the database? If yes, summarize. If not, based on the course material, what changes can you suggest to improve?

Checking all the relationships and the ERD, the relationships seem to be stated correctly, (ie. 1 customer can have many orders, 1 order can have many parts, etc). There is 1 M:N relationship which when read out, makes sense logically.

Is there consistency in a) naming between overview and entity/attributes b) entities plural, attributes singular c) use of capitalization for naming? If yes, summarize. If not, based on the course material, what changes can you suggest to improve?

I am a little confused on where the "manufacturingMethodID" attribute under the "Orders" entity is coming from. Since it is a FK, and the relationship is with "Part" I would expect to see a attribute associated with it but there isn't one. There is a "methodID" attribute under the "Machine" entity, but this is not a direct relationship. I may have understanding incorrect but if not, this makes me think with this attribute, there should be a relationship between "Orders" and "Machine".

ALSO, why is there two "partID" attributes under the "Part" entity? The first one makes sense as the PK but there is also a "partID" labeled as a PK which should correspond to a relationship like "colorID". I see, "colorID" and "materialID" but no "methodID", perhaps this accidental "partID: int, FK" was supposed to be "methodID"?

this is also on the ERD

(Bold text; taken from the assignment page on canvas

CS340 Staff (4/13/2026) *Project Step 1 Review on Canvas* [Answer Guidance Bullet Points]

<https://canvas.oregonstate.edu/courses/2042369/assignments/10464660>

Unbold text; all feedback thought of and written by me, Hunter.)

Samuel Nguyen^{4w}

Hello Kaiden and Daniel!

First off, I liked how you guys did a cover page, which makes it seem more professional. I liked what you guys did for your overview, which basically talks about how your database will track the 3D printing orders and the specific parts and materials that will be shipped/used. One tiny critique I would have is that maybe include the list of entities that are going to be used in your Overview, so that people already have an idea of what it will look like before heading to your next section below. I also think, like Jacob in the replies here, that bolding your entity names for each entity section makes them stand out more so that there is more contrast from each one. For revenue, I would personally use decimal as I would guess that not everything would be rounded up (for example, shipping fees or taxes added on above). Aside from that, I liked how you used an actual ERD website to design your ERD, since it describes what you guys said for the relationship very well. It makes it look professional, and I like how simple it is.

Tiernan Flanagan-Caldwell^{1mth}

Overview: Clearly states goal of project, which is to help your small startup scale up and use customer information to make better business decisions. For example trends in materials or colors can help inform your purchases. I think your proposal does a good job of addressing issues a business could have.

Entities: Your diagram and outline appear correctly formulated, as well as the relationships. E.g. One customer to many orders, one color/material to many parts. But here are some suggestions/nitpicks I found. Phone numbers shouldn't be stored as ints, if a number has a leading zero it would be deleted, or some numbers will overflow the int size limit, so I would use varchar ([source](#)). I would also look at using int for revenue, as I'm assuming you won't be rounding every order cost. Finally your naming conventions look good and are consistent. Overall I think you guys did a good job and are well prepared for the rest of the project. (Review work is original, sources cited)

Jacob Wahl1mth

Hey Daniel and Kaiden,

Your draft is very well done. The outline is clear and organized! I appreciate the camelCase in your attribute names.

A suggestion I have would be to bold the entity titles to clearly distinguish them from the others and make them stand out.

Another suggestion would be to tab down the "Entity-Relationship Diagram" text to put it on the next page. Same thing with references!

A technical change I would suggest is making Material and Color attributes inside the Part entity, rather than Color and Material existing on their own. If I am misunderstanding the representation of the Color and Material entity, feel free to disregard this. Or if you intend to track the stock of each color and material, I would add attributes to the Colors and Material to specify more information about them.

Overall, you clearly outline and create a functional ERD that describes the entities and their relationships well.

These are pretty nitpicky changes because your draft is very well done! Excellent work :)

(Review work is all my own)

Reply

Step 2 Feedback

Feedback by the TA

- To simplify the database, the team could consider whether **Colors** and **MaterialTypes** need to be separate entities. If they are only simple labels/descriptions, they could potentially be stored directly in **Materials**. Also, fields like **mostUsedMaterial**, **mostUsedColor**, **totalOrders**, and **totalSpent** in **Customers** appear to be derived values, so they could be calculated from order data instead of stored directly.
- For your DDL, how will you handle anomalies created by deleting data? Most of your foreign keys do not specify **ON DELETE** behavior. Please consider whether deleting related records should be restricted, cascaded, or handled another way. See Exploration - MySQL Cascade: https://canvas.oregonstate.edu/courses/2042369/pages/exploration-mysql-cascade?module_item_id=26640179.
- Please make sure to include all verbatim feedback in the PDF submission as required in the project instructions, even if it increases the length of the document.
- You haven't described any M:N relationship yet although you have junction table, it was given as a feedback in earlier submission, this will cause the reduction of marks in upcoming submission.

We are keeping Colors and MaterialTypes to improve normalization, as without it we would have many copies of the descriptions, as for example, there can be dozens of colors per materialType, and stating the materialType as a description every time would be inefficient for each of those combinations of one materialType and a dozen colors. Additionally, if additional parameters are wanted later on, this allows for that flexibility without significant reworking. Material allows for only unique combinations of these materialTypes and colors, to allow for stock keeping of how much of each combo is available. Each material can be in a M:M relationship with a part, so an additional junction table is required. WE will be including all feedback from the whole project as instructed now.

Student Feedback

- Yellow highlights are edits that need to be made and incorporated into the action plan.
- Green highlights are edits that we are choosing to ignore and need to be presented in the why we chose not to revise section.
- Purple edits are ones that we do not fully understand

Phoenix Jones2d

Hey group 14;

I really like the "3D printing as a service" concept! It's a very practical use case for a database, and your normalization section clearly explains how you moved from a redundant spreadsheet style to a more relational 3NF structure.

The schema mostly presents a physical model that follows the database outline and ER logical diagram, but there are a few exceptions. The ERD and SQL DDL include complex relationship links that aren't fully reflected in the outline. Specifically, the junction tables PartMachines and PartMaterials in the SQL use a massive amount of composite foreign keys that aren't listed in your written outline.

As for the consistency, there are some discrepancies there too. The outline and early tables use clean camelCase, but other tables like PartMaterials begin to use snake case for their attributes, making the DDL a little hard to read.

The schema is generally easy to read, but some of the long compound names make it a little hard on the eyes.

The tables form proper M:N relationships, but there are too many primary keys in the DDL, namely PartMaterials which has a whopping 7 keys which seems a little redundant. Usually you just need the IDs of the two tables you are joining.

Sample data looks good in the DDL, but it's not in the PDF where it should be.

The DDL itself looks a little off, there are parts that say "cite: 12" at random spots after lines that will cause the sql to not run. If these are supposed to be comments you should comment them out with the double dash.

A few discrepancies between DDL and pdf: The outline says colorID is a VARCHAR(30), but the SQL makes it an INT AUTO_INCREMENT. The outline marks several fields as UNIQUE or NOT NULL that didn't make it into the final SQL table definitions.

The DDL structure is pretty good and makes sense save for the "cite" thing. overall a good start, but there are definitely some improvements to be made.

This review is my own work.

Carter Kittell2d

Hi Kaiden, Daniel.

I like the level of detail recorded in your database. Seems like a realistic set up for a small manufacturing company, especially the part about machine scheduling/details.

Does the schema present a physical model that follows the database outline and the ER logical diagram exactly?

Yes, schema/ddl, outline, and ERD all match up for the most part, until PartMachines and Part materials/Materials tables. Same names as outline, but variables have been appended with longer names to signify foreign and primary keys from other tables.

Is there consistency in a) naming between overview, outline, ER and schema entity/attributes b) entities plural, attributes singular c) use of capitalization for naming?

Mix of Snake_case and CamelCase makes it somewhat difficult to read the schema, naming is more consistent in the outline. PartMachines/PartMaterials/Materials start using snake_case with long compound names for PK/FK while previous FK/PKs are still camelCase.

In the actual DDL, the long compound names make readability more difficult. if you know SQL it is pretty easy to figure out to tell what foreign keys are from what table based on the REFERENCES statements. You should change all your fk/pks in these sections to shorter camelCase references like your other tables for consistency with the rest of the schema. Also makes stuff less annoying to type when you are coding later. You can always include those titles as explanations in the notes sections of your code.

Also some of your attributes are using reserved names, and require a `` in order to work properly when being referenced or created.

Is the schema easy to read (e.g. diagram is clear and readable with relationship lines not crossed)?

Are intersection tables properly formed (e.g. two FKs and facilitate a M:N relationship)?

Relationships are clearly defined based on the ERD, outline no crossed lines, cardinality is obvious. PartMachines. Relationship for Machines section is unclear (Relationship: 1:M Relationship with Part_has_Machine), you don't have a table called Part_has_Machine. Multiple PKs on some tables, while compound primary keys have a purpose, I don't think you need 3-7 primary keys per table in some cases for a 1:m relationship, as I will explain in the next section.

Does the sample data suggest any non-normalized issues, e.g. partial dependencies or transitive dependencies?

In the SQL, are the primary and foreign keys correctly defined when compared to the Schema? Are appropriate CASCADE operations declared?

Yes. Sample data was not included in the PDF, but was included in the SQL DDL. Odd

Customers table has a major transitive dependency since it's reliant on pulling data from other tables. Since there is no CASCADE on delete, the fields in those original tables don't change if a customer is deleted, leaving data stale in those fields.

Large COMPOSITE pks specifically are causing potential redundancy issues in the junction tables, most of the PRIMARY KEYS in PartsMaterial are not necessary for the table. Every time you make an insert, you are reinserting the same data into multiple tables, so this is not 3NF compliant.

Is the SQL file syntactically correct?

Left some citations in the code uncommented, breaking the SQL. Otherwise syntax/structure is decent, and creates tables properly except for the above issues with design around relationships/redundancy

In the SQL, are the data types appropriate considering the description of the attribute in the database outline?

Many inconsistencies between the DDL and the outline. Lots of attributes stated in the outline as having constraints are missing them in the SQL. Too many to go over specifically.

In the SQL, are relationship tables present when compared to the ERD/Schema?

In the SQL, is all example data shown in the PDF INSERTED?

Is the SQL well-structured and commented (e.g. hand authored) or not (e.g. exported from MySQL)?

SQL is well structured except for the above issues with FK/PKs. Relationships appear to be implemented as is defined by the ERD in SQL, but with redundancy issues and a transitive dependency on the Customers Table, means they will need some work to function properly.

Data appeared to insert properly, however data is not copied into the PDF outline file, only shows up in the SQL DDL.

As a reviewer, clearly describe to what extent your feedback to the team was original (e.g. "all my work") or non-original (e.g. used AI tools per [Code Citation Tips](#)).

This review is all my own work.

Andrew Chung^{4d}

I like the idea of 3D printing as a service! With that said, here is my review:

- Does the schema present a physical model that follows the database outline and the ER logical diagram exactly?

No, I can see that the group might have confused with the difference between Schema and ER logical diagram. However, the outline and ER diagram are compatible. I think group 14 should add a Schema after the ER diagram.

- Is there consistency in a) naming between overview, outline, ER and schema entity/attributes b) entities plural, attributes singular c) use of capitalization for naming?

(a) No, the names are not the same from outline to the ER, and missing Schema to compare. When comparing outline to ER, we see same names for the entities, but we see attributes differing in consistency. In the outline, the attributes are camelCase while in the ER we see snake_case. Most notably, PartMaterials entity in the ER has attributes Parts_partID, Parts_Orders_orderID, Part_Order_Customer_customerID, Materials_materialID, Material_MaterialType_materialTypeID, Materials_Colors_colorID. The names are also confusing and needs to drop the the front snake_case.

(b) Despite the missing Schema, outline has all tables/entities names following PascalCase and are plural (Customers, Orders, Parts, PartMaterials, Materials, Colors, MaterialTypes,

PartMachines, Machines). Attributes follow singular names. An example is from Machines table/entity where we observe the attribute machineID, lastServiceDate, description. These also follow camelCase.

(c) as mention in part (b), we observe the usage of PascalCase for entity/table and camelCase for attributes when dealing with capitalization for naming.

- Is the schema easy to read (e.g. diagram is clear and readable with relationship lines not crossed)?

No, there is no schema. However, judging from the ER diagram, I think the group will make a great schema. There are no lines/relationships that cross over each other. I can also clearly identify the cardinality of relationships. An example, Customers to Colors is M:1 and optional. However, I will get back to this topic at the very end with a suggestion!

- Are intersection tables properly formed (e.g. two FKs and facilitate a M:N relationship)?

Yes/Maybe, I can see a clear PartMachines intersection table. The PartMachines table has FKs from Machines and Parts. These FKs can be found inside the intersection table as Parts_partID and Machines_machineID. However, my concern is that there are too many other FKs besides the FKs of the entity tables that are directly adjacent to this table. This same concern is present in PartMaterials too.

- Does the sample data suggest any non-normalized issues, e.g. partial dependencies or transitive dependencies?

There is no sample data, group 14 needs to add sample data of each table consisting of at least 3-5 rows.

- Is the SQL file syntactically correct? This can be easily verified by using PhPMyAdmin and your CS 340 database (do not forget to take backup of your own database before you do this!)

No, it is not syntactically correct. Throughout the SQL file, we see citations such as [cite: 12], that should be left as comments with -- or /* */. Because they are not commented out, the sql code fails.

- In the SQL, are the data types appropriate considering the description of the attribute in the database outline?

The data types in the SQL code is not the same as the attributes in the database outline. One example is in the Colors table, colorID has datatype INT NOT NULL AUTO_INCREMENT, but in the outline it has VARCHAR(30), UNIQUE, not NULL, PK written. There are many more and group 14 should check and change it so that both are consistent.

- In the SQL, are the primary and foreign keys correctly defined when compared to the Schema? Are appropriate CASCADE operations declared?

No, I cannot make the comparison because there is no Schema. The SQL file follows the naming convention of the ER Diagram but not the Outline. Just from looking at the SQL code, primary and foreign keys are defined appropriately. There are no CASCADE operations defined.

- In the SQL, are relationship tables present when compared to the ERD/Schema?

No, but looking at the ERD and SQL, the relationship tables PartMachines and PartMaterials exist. However it is confusing what the relationships the the relationship tables are trying to facilitate. An example of PartMachines, this entity table has partMachineID as the PK in the ERD, but in the SQL code it has PRIMARY KEY (partMachinesID, Parts_partID, Parts_Orders_orderID, Part_Order_Customer_customerID, Machines_machineID). These are way too many primary keys. Only 1 PK should exist per entity table as suggested on the ERD diagram (the yellow key icon).

- In the SQL, is all example data shown in the PDF INSERTED?

No, example data section is missing. The group should add example data. I recommend using the SQL tab of PhPMyAdmin to create, insert, and view the tables. It's a little faster than the VScode setup.

- Is the SQL well-structured and commented (e.g. hand authored) or not (e.g. exported from MySQL)?

No, as mentioned above, the citations needs to be commented. However, the clear division of creating tables and inserting data is well structured.

- As a reviewer, clearly describe to what extent your feedback to the team was original (e.g. "all my work") or non-original (e.g. used AI tools per [Code Citation Tips](#)).

This review is all my work!

Overall, I think the project is promising. Some suggestions I want to make is removing the snake_case naming in the ERD because I think it is plenty clear where the ids are coming from. I also suggest simplifying the relationships of the ERD diagram. One is the Colors entity table. I think it should be separated into ColorsPreferences and ColorsMaterials. These would behave as category tables for Customers and Materials. Currently, I can see that the Colors and MaterialTypes act as a shared Categories table, but as the lines suggest a optional relationship for the Customers to the Colors and MaterialTypes, these could be named as ColorsPreferences and MaterialTypesPreferences. These could have the options like "red", "blue", "green", and "no preference" as an example for Colors.

On the topic, I think the intersection tables have too many FKs. In the table PartMaterials, I don't think it needs id from color, order, customer, and materialType. It just needs the FKs from it's adjacent entity tables.

Shahzil Hussain Baloch6d

Hi Group 14,

Your database idea is clear and practical. The 3Designs and Manufacturing system has a strong business purpose because it connects the database to real operations like customers, orders, parts, materials, colors, and machines. I also liked the normalization section because it explains why the original design had repeated customer/order/part information and how separating the data into different tables helps reduce redundancy and avoid update or delete problems.

One area I would strengthen is the Step 2 deliverables inside the PDF. I can see the feedback section, updated outline, normalization explanation, and ERD, but I would double-check that the final PDF includes a clear schema diagram and example data for every table. The assignment asks for sample rows from each table, so adding 3-5 rows for Customers, Orders, Parts, PartMaterials, Materials, Colors, MaterialTypes, PartMachines, and Machines would make the relationships easier to verify. I would also do a consistency pass between the outline and the SQL file. A few names/data types seem different between the PDF and SQL. For example, the outline uses `mostUsedMaterial`, while the SQL uses `mostUsedMaterialType`. The outline also describes `colorID` and `materialTypeID` as `VARCHAR(30)` in some places, but the SQL appears to define them as integer auto-increment IDs. Also, the outline says Orders has `PartID`, while the SQL structure seems to place the order reference in Parts instead. These may be intentional design choices, but the PDF and SQL should match exactly so the schema is easy to grade and import.

One technical issue I would fix in the SQL file is the citation markers after some INSERT statements, such as `[cite: ...]`. Those should be removed from the `.sql` file because they are not valid SQL and could cause import errors in phpMyAdmin. The references can stay in the PDF, but the SQL file should only include valid SQL comments, CREATE TABLE statements, INSERT statements, and transaction/foreign-key setup.

One design point to consider is whether fields such as `totalOrders`, `totalSpent`, `mostUsedColor`, and `mostUsedMaterialType` should be stored in the Customers table. Since those values can usually be calculated from Orders and related tables, storing them may create update problems later unless there is a clear process for keeping them accurate.

This is a strong start with a clear business purpose. For the final version, I would focus on making the required Step 2 pieces easier to verify and ensuring the report, schema, and SQL all line up cleanly.

Actions based on the feedback

Most of the feedback from the students were repetitions of the same issues. The most glaring issues with the step 2 draft were the issues in the code that prevented it from being functional. For example, in the Customers table, we edited typo MatserialTypes(materialTypeID) to MaterialTypes(materialTypeID). Additionally, a lot of feedback stated that there were comments in the code that were not properly formatted as comments, thus leaving the code not functional. We corrected those errors and checked functionality in the ENGR servers. The actions taken based on that are listed in the section below. The next important critique was that we did not understand the difference between an ERD and a schema diagram, leading us to have an incomplete assignment. To remedy this, we created an ERD diagram and inserted it into the outline

Part 3 Feedback

Feedback from TA

Feedback from students

Isaac Taylor2d

Hello! took me some time to realize that i needed to copy paste the link instead of just clicking on it.

I was able to access each table easily via the top bar navigator. There is one link that leads to nothing, "Materials"

I do not see the option to insert anything into any table.

Every table has the option to delete each individual row, as well as update the attributes.

Junction tables haven't been implemented so there is no way to update an M:M relationship.

As for any other suggestions, spacing the links so that they're discernable from each other in the top bar, and spacing the data in the table to match the table header are the first couple things that jump to mind.

Bryan Lozano Gutierrez2d

Does the UI utilize a SELECT for every table in the schema? In other words, data from each table in the schema should be displayed on the UI. Note: it is generally not acceptable for just a single query to join all tables and displays them.

The UI utilizes SELECT functionality for the tables in the schema. Separate pages were created for

Does the UI implement an INSERT form for at least one table in the schema? In other words, there should be UI input fields that correspond to at least one table.

The UI implements INSERT functionality for many entities through their edit pages. I understand that some parts are still incomplete, but even then the structure and layout for adding records looks well planned.

Does the UI have at least one DELETE for any one entity? In other words, is there a form/link/button that will allow the deletion of a row in at least one table?

The UI has at least one DELETE for every entity. I noticed trash can icons on multiple entity pages such as Customers, Orders, Parts, Machines, Colors, and Material Types, which efficiently show where the DELETE functionality resides in.

Does the UI have at least one DELETE that will remove things from a M:M relationship? In other words, if an order is deleted from the Orders table, it should also delete the corresponding rows from the OrderDetails table, BUT it should not delete any Products or Customers.

I have not fully seen the M:M relationship pages implemented yet. Based on the DML and project description, it looks like the plan is to handle the junction tables separately.

Is there at least one UPDATE form in the UI for any one entity? In other words, in the case of Products, can productName, listPrice, qtyOnHand, e.g. be updated for a single ProductID record?

There is at least one UPDATE form for each of the entities. The pen icons navigate to separate edit pages with editable form fields for records in all entities, except for the Materials entity, which does not currently have any data in its page.

Is there at least one UPDATE form in the UI to modify an M:M relationship? In other words, does the UPDATE allow the user to select a different foreign key value to update the intersection table with?

I have not yet seen an UPDATE form in the UI to modify an M:M relationship, but your post mentions that the junction tables are planned to be handled differently later on.

Do you have any other suggestions for the team to help with their HTML UI? For example, using AS aliases to replace obscure column names such as fname with First Name.

One suggestion I have is to add more spacing between the navigation links, as they currently are close together, almost appearing as if they were one link. The Materials page also currently only displays the link without table data yet, but I understand it is a work in progress. Another suggestion would be to maybe change the camelCase column names to improve readability, like changing "orderId" to "Order ID". Overall this is a strong draft, especially considering the Canvas outage issues.

As a reviewer, clearly describe to what extent your feedback to the team was original (e.g. "all my work") or non-original (e.g. used AI tools per Code Citation Tips).

All my work is original.

I wrote this entire review myself.

Tiernan Flanagan-Caldwell

Hey all! I believe I reviewed this project before so it's fun to see how it's updated.

- Does the UI utilize a SELECT for every table in the schema?

- *Yes I believe so. There is only one row of data but each of the selected tables show an appropriate row.*
- Does the UI implement an INSERT form for at least one table in the schema? In other words, there should be UI input fields that correspond to at least one table.
 - *Unless I'm missing something and the update/edit form will also function as an insert form there does not appear to be. Every table does have an edit form although, so it should be easy enough to implement.*
- Does the UI have at least one DELETE for any one entity? In other words, is there a form/link/button that will allow the deletion of a row in at least one table?
 - *Yes, every entity page does have a delete button*
- Does the UI have at least one DELETE that will remove things from a M:M relationship? In other words, if an order is deleted from the Orders table, it should also delete the corresponding rows from the OrderDetails table, BUT it should not delete any Products or Customers.
 - *I believe so? Every page has a delete button but I'm unsure of the implementation. Looking at the document there should be an intersection table for Parts-Machines and Parts-Materials. Although the post here says junction tables will be handled differently in the future.*
- Is there at least one UPDATE form in the UI for any one entity? In other words, in the case of Products, can productName, listPrice, qtyOnHand, e.g. be updated for a single ProductID record?
 - *Yes, every page has an edit button. One mistake though, in the colors page, you can edit the phone number of the color, which I think is kinda funny.*
- Is there at least one UPDATE form in the UI to modify an M:M relationship? In other words, does the UPDATE allow the user to select a different foreign key value to update the intersection table with?
 - *At this time junction tables are not implemented in the UI.*
- Do you have any other suggestions for the team to help with their HTML UI? For example, using AS aliases to replace obscure column names such as fname with First Name.
 - *It looks like you have laid a solid groundwork for your website. The only suggestions I have would be cleaning up the UI with a bit of styling, and implementing intersection tables, which you have already planned.*
- As a reviewer, clearly describe to what extent your feedback to the team was original
 - *Feedback is entirely original and my own.*

Fiona Dean

2 days ago

OK First off you made me feel so much better - my link to the website I ALSO accidentally mis-formatted in the SAME exact way you did!

So after giggling at our LITERAL same mistake, I copied (instead of clicking) your URL into my browser and boom! It works. Phew.

You got a little further than my group did and I can see how each page will be used. I can't wait to see the forms. I can see where you've put a trash can (a future delete button) and edit buttons and I can already edit things! Neat!

Actions Taken Based on Feedback

Corrected link issue in Ed Discussion, added M:N table editing, creation, and deletion. We will also style the site if time permits.

Actions taken outside of feedback

When testing the DDL in the ENGR servers, we ran into issues with two tables. First, the PartMaterials table was set to AUTO_INCREMENT. This led to me seeing 40,500 rows, when I did a SELECT * FROM on all the tables. Because of this, we changed the Orders table to use a single primary key. This was one of the suggestions in the feedback, but we could not visualize it until testing the code in the ENGR servers. After making these changes, we updated our outline and diagrams to reflect the changes. There were also other changes made to the DDL. In fact, it took three drafts to come up with a completed product.

Normalization

Prior to and in the first step of the project, group 14 had trouble with table separation. We did not have enough tables, so initially, if customers had more than one part, they would have their name, address, and other transaction-related information repeated for however many parts they purchased. This is a clear example of data redundancy. There were also examples of delete anomalies in places where if you deleted one record, it could delete the only color entry. For the first normalization step, we ensured that the keys like Customers, Orders, and Parts all had unique identifiers without repetition. This got us to 1NF. After this, we moved onto 2NF by getting rid of partial dependencies. Honestly, our group did not have to do much for this step at all. We simply separated the customer details from the transactions in the Orders table. The last portion of normalization for step 2 was getting to 3NF. Going from 2NF to 3NF was a lot more resource intensive. The definition of 3NF involves removing transitive dependencies, so we had to ensure Colors, MaterialTypes, and Machines were all in their own table. This way, the descriptive qualities of the partID's were not dependent on each other.

Customer	Address	Part	Color	Material	Date
Daniel	456 Culver Lane.	4inch Propellors	Black	PLA	12/31/2026
Haiden	123 Rainbow Rd.	Christmas Decoration	Red	TPU	1/15/2026
Chris	987 Carpet Door Drive	RC Dune Buggy Frame	Orange	PETG	3/4/2026

This is an example of non-normalized data. This image was generated using the snip tool to capture a sheet made using Microsoft Excel Online

Overview

This database backend solves the problem of manual order logistics for our firm, 3Designs and Manufacturing, by allowing us to track 3D printing orders more effectively. This firm is a small startup that currently makes 1000 sales per year. There are 600 customers a year, as many will complete their project and no longer need 3Design and Manufacturing's services. Each year 3Design and Manufacturing produces 600,000 dollars in annual income. The proposed system will track orders, including individual specifications for parts to be printed. These specifications will include broad categories such as Resin or Fused Deposition Modeling (FDM) (MicroCenterMaker, 2024). Further, the database will also contain specifics about what type and color of resin or plastic is used for an order. Because our organization is focused on order fulfillment, we will also include tables to track customer information and database transactions. An example of what a typical transaction could include is the mass and infill density of each part ordered, along with the quantity of ordered parts. This empowers our organization to perform data analytics and trend analysis with simple, yet systematic, SQL queries. For example, we will be able to identify which materials are most used. Using this information, we can make informed decisions on exactly what we should be purchasing more of in the future. This will also lead us to understand what should be kept in stock, which colors are most popular, and the size and scale of an average order. A strong central database that fulfills the ideas outlined above will help 3Designs and Manufacturing expand vertically and horizontally as a firm (Curry, n.d.-d).

Database Outline.

Customers: Customer details for 3Designs and Manufacturing.
customerID: INT, AUTO_INCREMENT, UNIQUE, NOT NULL, PK
email: VARCHAR(45), NOT NULL
phoneNumber: VARCHAR(45), NOT NULL
primaryContact: VARCHAR(45), NOT NULL
address: VARCHAR(180), NOT NULL
Relationship: A 1:M relationship with Orders.

Orders: Order information containing financial and date details.

orderID: INT, AUTO_INCREMENT, NOT NULL, PK

revenue: DECIMAL(19,2), NOT NULL

customers_customerID: INT, FK, NOT NULL

date: DATE, NOT NULL

Relationship: A M:1 relationship with Customers. A 1:M relationship with Parts.

Parts: Contains technical manufacturing specifications for specific items.

partID: INT, AUTO_INCREMENT, UNIQUE, NOT NULL, PK

partName: VARCHAR(45), NOT NULL

partPath: VARCHAR(180), NOT NULL

quantity: INT, NOT NULL

mass: INT, NOT NULL

infillDensity: TINYINT, NOT NULL

orders_orderID: INT, FK, NOT NULL

Relationship: A M:1 relationship with Orders. Participates in M:N relationships with Materials and Machines via junction tables.

Materials: Inventory tracking for raw 3D printing supplies.

materialID: INT, AUTO_INCREMENT, UNIQUE, NOT NULL, PK

kilograms: INT, NOT NULL

materialTypes_materialTypeID: VARCHAR(45), FK, NOT NULL

colors_colorID: VARCHAR(45), FK, NOT NULL

Relationship: A M:1 relationship with MaterialTypes and Colors. Participates in a M:N relationship with Parts.

Colors: Standardized color catalog for inventory and preferences.

colorID: VARCHAR(45), UNIQUE, NOT NULL, PK

colorDescription: VARCHAR(45), NOT NULL

Relationship: A 1:M relationship with Materials

MaterialTypes: Categorization of printing materials (e.g., PLA, PETG).

MaterialTypeID: VARCHAR(45), UNIQUE, NOT NULL, PK

materialTypeDescription: VARCHAR(45), NOT NULL

Relationship: A 1:M relationship with Materials

Machines: Inventory of 3D printers and maintenance tracking.

machineID: INT, AUTO_INCREMENT, UNIQUE, NOT NULL, PK
machineDescription: VARCHAR(45), NOT NULL
lastServiceDate: DATE, NOT NULL
Relationship: Participates in a M:N relationship with Parts via PartMachines.

PartMaterials (Junction Table): Facilitates the M:N relationship between Parts and Materials.

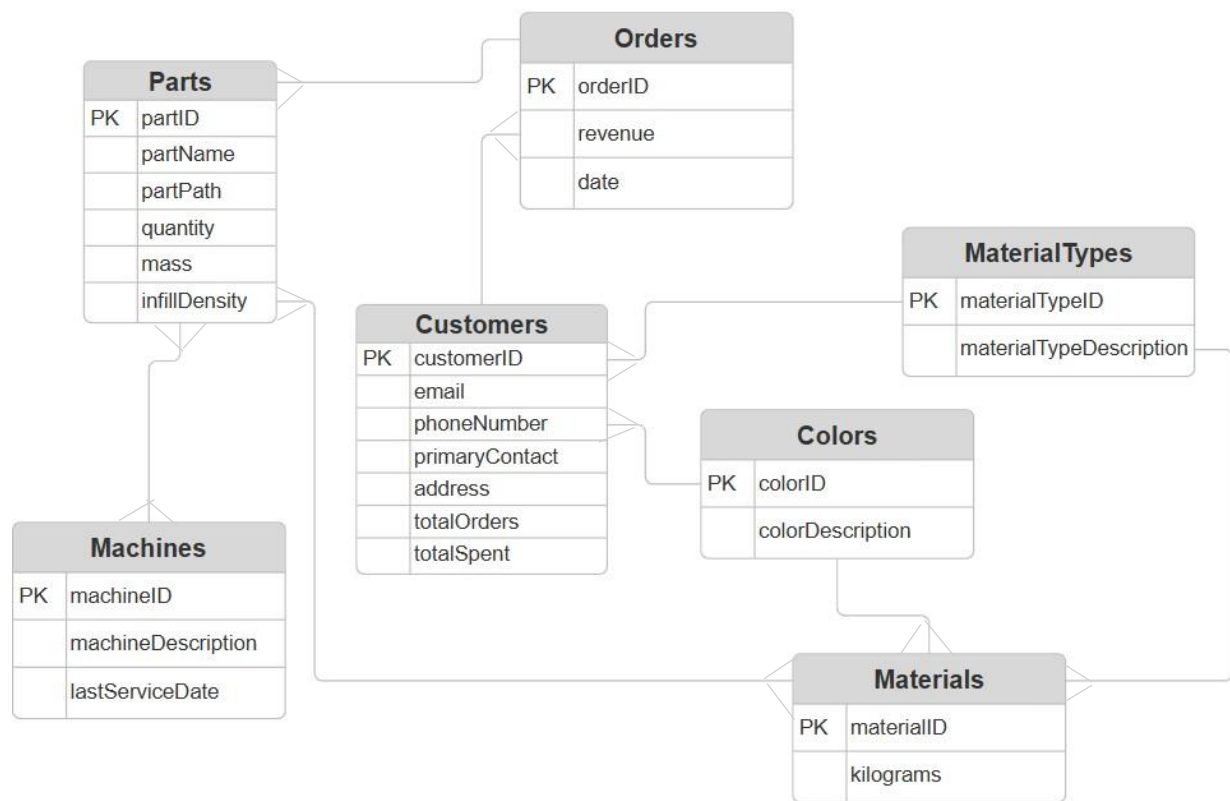
partMaterialsID INT, AUTO_INCREMENT, UNIQUE, NOT NULL, PK
parts_partID: INT, FK, NOT NULL
materials_materialID: INT, FK, NOT NULL
Relationship: Used dedicated PK while facilitating M:N relationship between Parts and Materials.

PartMachines (Junction Table): Uses dedicated PK and facilitates the M:N relationship between Parts and Machines.

partMachinesID: INT, AUTO_INCREMENT, UNIQUE, NOT NULL, PK
parts_partID: INT, FK, NOT NULL
machines_machineID: INT, FK, NOT NULL
Relationship: Connects Parts and Machines to track which equipment was used for which part.

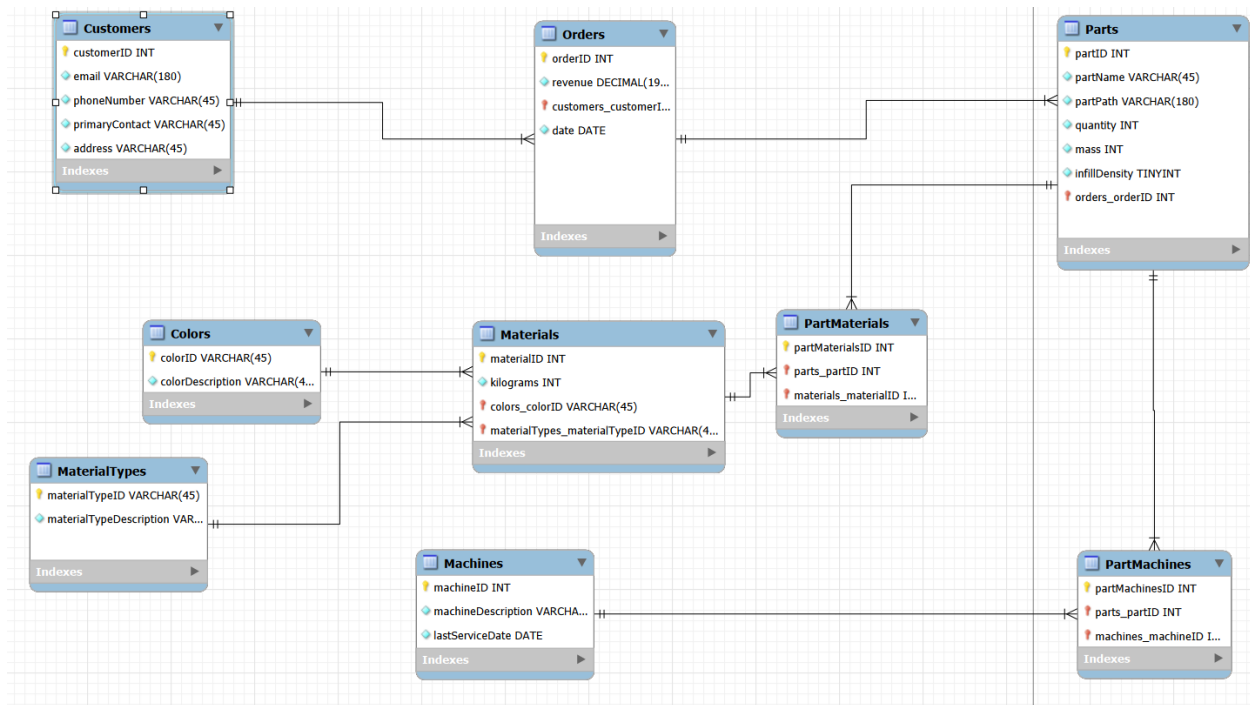
Note: The information used to create the outline above was obtained from the Module 2 learning Resources (Curry, n.d.-b,-c).

Entity-Relationship Diagram



This is an Entity-Relationship Diagram that presents the information detailed in the outline above. It was created using the Module 2 learning materials (Curry, n.d.-a).

Schema Diagram



Test data

Test data is included as a .xlsx file in the zip, for easier readability.

References

- Curry, M. (n.d.-a). *Exploration - creating ER diagram MySQL workbench*. Canvas. Oregon State University. https://canvas.oregonstate.edu/courses/2042369/pages/exploration-creating-er-diagram-mysql-workbench?module_item_id=26640141
- Curry, M. (n.d.-b). *Exploration - database design patterns*. Canvas. Oregon State University. https://canvas.oregonstate.edu/courses/2042369/pages/exploration-database-design-patterns?module_item_id=26640137

Curry, M. (n.d.-c). *Exploration - the ER model*. Canvas. Oregon State University.
https://canvas.oregonstate.edu/courses/2042369/pages/exploration-the-er-model?module_item_id=26640139

Curry, M. (n.d.-d). *Project step 2: Database outline*. Canvas. Oregon State University.
https://canvas.oregonstate.edu/courses/2042369/assignments/10464658?module_item_id=26640144

MicroCenterMaker. (2024, April 26). *FDM vs. resin 3D printing: Which is right for you?* Micro Center. <https://www.microcenter.com/site/mc-news/article/fdm-vs-resin.aspx>